

Monte Carlo Tree Search

Introducción rigurosa desde cero

Este documento presenta el algoritmo Monte Carlo Tree Search (MCTS) partiendo de cero, enunciando explícitamente las hipótesis en cada paso, e incluyendo referencias a recursos externos curados por nivel.

Contenidos:

- El problema de decisión secuencial
- El bandido multi-brazo y UCB1
- Las cuatro fases de MCTS
- UCT: Upper Confidence Bounds for Trees
- Garantías teóricas de convergencia
- Pseudocódigo comentado
- Cuándo se cumplen/violan las hipótesis
- Recursos por nivel de profundidad

Índice

1. ¿Qué problema queremos resolver?	2
2. Prerrequisito: el problema del bandido multi-brazo	2
2.1. Planteamiento	2
2.2. El arrepentimiento (regret)	3
2.3. El algoritmo UCB1	3
3. De bandidos a árboles de decisión	4
3.1. El árbol de juego	4
3.2. La conexión entre árboles y bandidos	5
4. El algoritmo MCTS: las cuatro fases	5
4.1. Fase 1 Selección (UCT)	5
4.2. Fase 2 Expansión	6
4.3. Fase 3 Simulación (Rollout)	6
4.4. Fase 4 Retropropagación (Backpropagation)	7
4.5. Decisión final	7
5. Garantías teóricas de convergencia	8
6. Pseudocódigo comentado	8
7. Cuándo se cumplen y cuándo se violan las hipótesis	10
8. Recursos por nivel de profundidad	11
9. Resumen: el mapa conceptual del algoritmo	12
Referencias principales	13

1. ¿Qué problema queremos resolver?

Imagina que juegas al ajedrez. En cada turno tienes que elegir un movimiento. El problema es que el árbol de posibilidades futuras es astronómico: el ajedrez tiene aproximadamente 10^{120} partidas posibles (el llamado **número de Shannon**). Ningún computador puede explorar todo ese árbol en tiempo útil.

Durante décadas la solución fue **minimax con poda alfa-beta**: explorar el árbol hasta cierta profundidad y usar una función de evaluación escrita a mano (p.ej., ¿el caballo vale 3 peones?). Esto funcionó razonablemente bien para ajedrez, pero fracasó en **Go**: el tablero es más grande, el factor de ramificación es ≈ 250 , y nadie supo escribir una buena función de evaluación para Go durante décadas.

MCTS nació precisamente para resolver este vacío:

Idea clave

¿Cómo tomar buenas decisiones cuando el árbol es demasiado grande para explorarlo exhaustivamente *y* no disponemos de una función de evaluación de posiciones?

La respuesta de MCTS es **muestrear el futuro aleatoriamente** (método Monte Carlo) y **guiar la búsqueda** hacia las ramas más prometedoras usando estadísticos acumulados. El resultado es un algoritmo que:

- No necesita función de evaluación escrita a mano.
- Mejora su calidad cuanto más tiempo de cómputo se le da (*anytime*).
- Converge a la decisión óptima en el límite.

2. Prerrequisito: el problema del bandido multi-brazo

Antes de entender MCTS es imprescindible entender el problema matemático que reside en su corazón.

2.1. Planteamiento

Situación: Estás en un casino con K máquinas tragaperras (los *ñbrazos* del bandido). Cada máquina $i \in \{1, \dots, K\}$ tiene una distribución de recompensa ν_i con media desconocida μ_i . Puedes tirar una palanca a la vez. Quieres **maximizar la recompensa total** en n tiradas.

Hipótesis H1

Las recompensas de cada máquina son **independientes entre sí e idénticamente distribuidas** (i.i.d.) a lo largo del tiempo *para esa máquina*. Es decir, la distribución de la máquina i no cambia con el tiempo ni depende de las otras máquinas.

Hipótesis H2

Las recompensas están **acotadas** en el intervalo $[0, 1]$ (o en algún intervalo conocido $[a, b]$). Esta hipótesis es necesaria para obtener cotas de concentración (desigualdad de Hoeffding) que garantizan los resultados teóricos.

El dilema fundamental se llama **equilibrio exploración–explotación**:

- **Explotación:** tiramos siempre de la máquina que mejor nos ha ido hasta ahora.
- **Exploración:** probamos máquinas poco visitadas por si son mejores.

Si solo explotamos, podemos quedarnos atascados en un óptimo local. Si solo exploramos, desperdiciamos tiradas en máquinas malas.

2.2. El arrepentimiento (regret)

Para medir la calidad de un algoritmo de bandido, definimos el **arrepentimiento acumulado** tras n tiradas:

$$R_n = n\mu^* - \sum_{t=1}^n r_t \quad (1)$$

donde $\mu^* = \max_i \mu_i$ es la recompensa de la mejor máquina y r_t es la recompensa obtenida en la tirada t . Un algoritmo es bueno si R_n crece lentamente con n .

Resultado fundamental (Lai & Robbins, 1985)

Bajo las hipótesis H1 y H2, ningún algoritmo puede lograr regret menor que $\Omega(\log n)$ en general. El regret logarítmico es por tanto **óptimo**.

2.3. El algoritmo UCB1

El algoritmo **UCB1** (Auer et al., 2002) elige en cada tirada t la máquina i que maximiza:

$$\text{UCB1}(i) = \underbrace{\bar{x}_i}_{\text{explotación}} + \underbrace{\sqrt{\frac{2 \ln n}{n_i}}}_{\text{exploración}} \quad (2)$$

donde:

- $\bar{x}_i = \frac{1}{n_i} \sum_{\text{tiradas en } i} r_t$: media empírica de recompensas de la máquina i .
- n : total de tiradas realizadas hasta el momento.
- n_i : número de veces que se ha tirado la máquina i .

Interpretación geométrica del segundo término: El término $\sqrt{2 \ln n / n_i}$ es proporcional al *radio de un intervalo de confianza* para μ_i . Cuando n_i es pequeño (máquina poco explorada), este término es grande, incentivando visitar esa máquina. A medida que n_i crece, el intervalo se estrecha y confiamos más en la estimación empírica.

Garantía de UCB1 (Auer et al., 2002)

Bajo las hipótesis H1 y H2, UCB1 garantiza un regret acumulado:

$$\mathbb{E}[R_n] \leq \sum_{i: \mu_i < \mu^*} \frac{8 \ln n}{\Delta_i} + \left(1 + \frac{\pi^2}{3}\right) \sum_{i: \mu_i < \mu^*} \Delta_i$$

donde $\Delta_i = \mu^* - \mu_i$ es la brecha de la máquina i respecto a la óptima. En particular, $\mathbb{E}[R_n] = O(\log n)$, que es óptimo.

3. De bandidos a árboles de decisión

3.1. El árbol de juego

Hipótesis H3

El problema puede modelarse como un **árbol de búsqueda** donde cada nodo representa un estado del juego, cada arista representa una acción legal, y existen estados terminales con recompensa definida (+1 victoria, 0 derrota, 0,5 empate).

Hipótesis H4

El árbol es **demasiado grande** para explorarlo exhaustivamente. Por esto no podemos aplicar minimax completo.

Hipótesis H5

El juego es de **información perfecta**: ambos jugadores conocen el estado completo del sistema en todo momento. Esta hipótesis se relaja en versiones avanzadas (ver POMCP).

Formalmente, modelamos el problema como un **proceso de decisión de Markov** (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

donde $\mathcal{S}$ es el espacio de estados, $\mathcal{A}$ el espacio de acciones, $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ la función de transición, R la función de recompensa, y $\gamma \in [0, 1]$ el factor de descuento.

3.2. La conexión entre árboles y bandidos

Idea clave

Cada nodo del árbol tiene varios hijos (movimientos posibles). Elegir cuál hijo visitar es **exactamente el problema del bandido multi-brazo**: cada hijo es una “máquina” cuya recompensa esperada es la calidad de ese movimiento, que desconocemos.

La idea central de MCTS es **tratar cada nodo interno del árbol como una instancia independiente del problema del bandido**.

Esta observación, aparentemente simple, es la clave que hace funcionar MCTS. Cada nodo almacena dos estadísticos:

- $N(v)$: número de veces que el nodo v ha sido visitado.
- $W(v)$: suma acumulada de recompensas obtenidas en simulaciones que pasaron por v .

El **valor estimado** del nodo es $Q(v) = W(v)/N(v)$.

4. El algoritmo MCTS: las cuatro fases

MCTS construye iterativamente un árbol $\mathcal{T}$ partiendo de la raíz (estado actual del juego). Cada iteración ejecuta cuatro fases.

4.1. Fase 1 Selección (UCT)

Hipótesis en uso: H1, H2, H3, H4.

Partiendo de la raíz, recorremos el árbol hacia abajo usando el criterio **UCT** (Upper Confidence Bounds for Trees, Kocsis & Szepesvári, 2006), que adapta UCB1 al contexto de árboles:

$$\text{UCT}(v) = \underbrace{\frac{W(v)}{N(v)}}_{\text{explotación}} + c \underbrace{\sqrt{\frac{\ln N(\text{padre}(v))}{N(v)}}}_{\text{exploración}} \quad (3)$$

donde $c > 0$ es el parámetro de exploración. El valor teóricamente motivado es $c = \sqrt{2}$, aunque en la práctica se ajusta por validación cruzada.

Procedimiento: Desde la raíz, en cada nivel seleccionamos el hijo con mayor $UCT(v)$, y descendemos hasta alcanzar un **nodo hoja**: un nodo que tiene al menos un hijo no visitado, o un estado terminal.

Nota

£**Por qué $\ln N(\text{padre})$ y no $\ln n$ total?** Porque lo que nos importa es el número de veces que hemos tenido la oportunidad de elegir entre los hijos de ese nodo. Esto hace que el criterio sea localmente justo en cada nivel del árbol.

4.2. Fase 2 Expansión

Hipótesis en uso: H3.

Una vez en el nodo hoja, si el nodo no es terminal y tiene hijos no visitados, añadimos **uno de esos hijos** al árbol, inicializado con $N = 0$, $W = 0$.

Hipótesis H6

Expandir **un único hijo** por iteración (en lugar de todos) es suficiente para garantizar la convergencia y mantiene el coste por iteración acotado por $O(b)$, donde b es el factor de ramificación.

En algunas implementaciones se expanden todos los hijos a la vez (expansión total). Esto acelera la convergencia en árboles con branching factor pequeño, pero puede ser prohibitivo si b es grande.

4.3. Fase 3 Simulación (Rollout)

Hipótesis en uso: H1, H2, H3, y las nuevas H7 y H8.

Esta es la fase más característica y la que da el nombre al Monte Carlo al algoritmo.

Hipótesis H7

Desde el nodo expandido, podemos **simular partidas** hasta el final usando una política simple (típicamente movimientos uniformemente aleatorios). El resultado de esa partida es una **estimación insesgada** del valor real del nodo:

$$\mathbb{E}[\hat{G} \mid s] = V^{\pi_{\text{ro}}}(s)$$

donde π_{ro} es la política de rollout.

Hipótesis H8

La **varianza** de las recompensas es finita. Esto garantiza que el promedio de múltiples rollouts converge al valor esperado por la ley de los grandes números.

Nota

La hipótesis H7 es **fuerte**: un rollout aleatorio es una estimación muy ruidosa del valor real. Funciona porque se promedia sobre *muchas* iteraciones. En juegos donde la aleatoriedad no correlaciona con la victoria (p.ej., Go a nivel profesional), el rollout aleatorio es casi inútil; esto motivó reemplazarlo por redes neuronales en AlphaZero.

El resultado es un escalar $\hat{G}$:

- $\hat{G} = 1$: el jugador actual ganó la partida simulada.
- $\hat{G} = 0$: el jugador actual perdió.
- $\hat{G} = 0,5$: empate.

4.4. Fase 4 Retropropagación (Backpropagation)

Hipótesis en uso: H3.

El resultado $\hat{G}$ se propaga hacia arriba actualizando todos los nodos del camino recorrido en la Fase 1:

$$N(v) \leftarrow N(v) + 1, \quad W(v) \leftarrow W(v) + \hat{G} \quad (4)$$

para cada v en el camino desde el nodo expandido hasta la raíz.

Nota

Juegos de dos jugadores: cuando $\hat{G}$ sube al nodo del oponente, se usa $1 - \hat{G}$. Lo que es bueno para el jugador actual es malo para el rival. Esta alternancia es automática si cada nodo almacena la perspectiva del jugador que mueve en ese turno.

4.5. Decisión final

Tras n iteraciones, se elige la acción desde la raíz hacia el hijo con **mayor** $N(v)$ (más visitado). Alternativamente, se puede usar mayor $Q(v) = W(v)/N(v)$.

Idea clave

¿Por qué más visitado en lugar de mayor Q ? Porque el hijo más visitado es aquel que UCT ha confirmado repetidamente como prometedor. Un hijo con Q alto pero pocas visitas puede ser un **artefacto estadístico** (error de muestreo con pocos datos).

5. Garantías teóricas de convergencia

Hipótesis en uso: H1, H2.

Teorema de convergencia de UCT (Kocsis & Szepesvári, 2006)

Sea a^* la acción óptima en la raíz. Bajo las hipótesis H1 y H2 (recompensas acotadas, i.i.d. por nodo), UCT satisface:

$$\Pr[\text{acción subóptima seleccionada tras } n \text{ iteraciones}] = O\left(\frac{\log n}{n}\right)$$

En particular, la probabilidad de error tiende a 0 cuando $n \rightarrow \infty$: UCT **converge a la decisión óptima**.

Propiedades adicionales:

- **Anytime:** MCTS puede detenerse en cualquier momento y devuelve la mejor decisión encontrada hasta ese punto. Cuanto más tiempo se le da, mejor.
- **Sin función de evaluación:** no requiere conocimiento experto del dominio más allá de conocer las reglas del juego.
- **Asimetría:** MCTS explora el árbol de forma asimétrica, profundizando más en las ramas prometedoras y abandonando rápidamente las malas. A diferencia de minimax, no explora en anchura uniforme.

Complejidad computacional por iteración:

Fase	Complejidad
Selección (profundidad d)	$O(d)$
Expansión (branching b)	$O(b)$
Simulación (profundidad h)	$O(h)$
Retropropagación	$O(d)$
Total por iteración	$O(d + b + h)$
Total (n iteraciones)	$O(n(d + b + h))$

6. Pseudocódigo comentado

```

1 def MCTS(estado_raiz, n_iteraciones, c=1.41):
2     """
3     Ejecuta n_iteraciones del ciclo MCTS y retorna la mejor
      accion.
4     c: parametro de exploracion (sqrt(2) por defecto).

```

```

5      """
6      raiz = Nodo(estado=estado_raiz, padre=None)
7
8      for _ in range(n_iteraciones):
9
10         # FASE 1: SELECCION
11         # Hipotesis: usamos UCT para navegar el arbol ya
            construido.
12         # Descendemos hasta un nodo hoja (no completamente
            expandido
13         # o estado terminal).
14         nodo = raiz
15         while nodo.completamente_expandido() and not nodo.
            es_terminal():
16             nodo = max(nodo.hijos,
17                         key=lambda v: v.W/v.N + c*(log(v.padre
18                             .N)/v.N)**0.5)
19
20         # FASE 2: EXPANSION
21         # Hipotesis: hay hijos no visitados; expandimos uno.
22         if not nodo.es_terminal():
23             nodo = nodo.expandir_hijo_no_visitado()
24
25         # FASE 3: SIMULACION (ROLLOUT)
26         # Hipotesis: el rollout aleatorio estima el valor
            real del nodo
27         # (estimacion insesgada pero ruidosa).
28         resultado = simular_partida_aleatoria(nodo.estado)
29
30         # FASE 4: RETROPROPAGACION
31         # Actualizamos N y W en todos los nodos del camino
            recorrido.
32         while nodo is not None:
33             nodo.N += 1
34             nodo.W += resultado
35             resultado = 1 - resultado # alternamos
                perspectiva (2 jugadores)
36             nodo = nodo.padre
37
38         # Elegir la mejor accion: hijo de la raiz con mayor N (
            mas visitado)
39     return max(raiz.hijos, key=lambda v: v.N).accion

```

Listing 1: MCTS pseudocódigo en Python

Estructura del nodo:

```

1 class Nodo:
2     def __init__(self, estado, padre, accion=None):
3         self.estado = estado          # estado del juego en
            este nodo
4         self.padre = padre            # nodo padre (None si es
            raiz)

```

```

5         self.accion = accion          # accion que llevo aqui
           desde el padre
6         self.hijos = []                # lista de nodos hijo
           expandidos
7         self.N = 0                     # numero de visitas
8         self.W = 0.0                   # suma acumulada de
           recompensas
9
10        def completamente_expandido(self):
11            acciones_legales = self.estado.acciones_legales()
12            acciones_expandidas = [h.accion for h in self.hijos]
13            return set(acciones_legales) == set(
                acciones_expandidas)
14
15        def expandir_hijo_no_visitado(self):
16            acciones_no_visitadas = [
17                a for a in self.estado.acciones_legales()
18                if a not in [h.accion for h in self.hijos]
19            ]
20            accion = random.choice(acciones_no_visitadas)
21            nuevo_estado = self.estado.aplicar(accion)
22            hijo = Nodo(estado=nuevo_estado, padre=self, accion=
                accion)
23            self.hijos.append(hijo)
24            return hijo

```

Listing 2: Clase Nodo para MCTS

7. Cuándo se cumplen y cuándo se violan las hipótesis

Hipótesis	¿Cuándo se viola?	Consecuencia y solución
H1 Recompensas i.i.d.	Oponente adaptativo; entornos no estacionarios; juegos con elementos aleatorios dependientes del tiempo	UCT puede no converger. Solución: usar variantes robustas o modelar explícitamente la estocasticidad.
H2 Recompensas acotadas	Recompensas continuas sin límite (p. ej., puntuación acumulable indefinidamente)	Las cotas de regret no aplican directamente. Solución: normalizar las recompensas al rango [0, 1].
H3 Árbol de juego	Juegos con ciclos (tablero infinito, posiciones repetidas en ajedrez/Go)	El árbol se convierte en un grafo (DAG). Solución: tablas de transposición (tabla hash de estados → estadísticos).

Hipótesis	¿Cuándo se viola?	Consecuencia y solución
H5 Información perfecta	Póker, bridge, Starcraft, robótica con sensores ruidosos	El estado real es latente (POMDP). Solución: POMCP (opera sobre espacios de creencias) o IS-MCTS (Information Set MCTS).
H6 Expansión de un hijo	Branching factor muy pequeño ($b \leq 5$)	La expansión total (todos los hijos a la vez) puede ser más eficiente.
H7 Rollout estima bien	Juegos donde el azar aleatorio no correlaciona con la victoria (Go a nivel profesional, ajedrez)	El rollout es inútil o contraproducente. Solución: reemplazarlo por una red neuronal de valor (AlphaZero).
H8 Varianza finita	Recompensas de cola pesada (distribuciones de Pareto)	La ley de los grandes números converge muy lentamente. Solución: algoritmos de bandido robustos (mediana de medias).

Cuadro 1: Resumen de hipótesis, condiciones de violación y soluciones.

8. Recursos por nivel de profundidad

Nivel 0 Intuición visual (sin matemáticas)

- **The Animated MCTS** Thomas Kurbiel, Medium: animaciones GIF de cada una de las cuatro fases. Ideal para construir la intuición visual antes de abordar la fórmula.
- **MCTS for Dummies** Medium: analogía con exploración de mapas y lenguaje completamente accesible.

Nivel 1 Introductorio con código Python

- **Int8 Beginner's Guide to MCTS**: el mejor recurso introductorio completo. Cubre teoría, pseudocódigo e implementación en Python con juego de ejemplo. **Altamente recomendado como primer recurso técnico.**
- **ai-boson MCTS Python Step by Step**: implementación limpia y comentada con Tic-Tac-Toe. Ideal para leer en paralelo al código.

Nivel 2 Introductorio con teoría básica

- **TDS MCTS An Introduction** Benjamin Wang: narrativa clara con fórmulas básicas y ejemplos de aplicación.

- **Mastering Reinforcement Learning MCTS Chapter**: libro de texto libre en línea con MCTS aplicado a RL, incluyendo GridWorld y Freeway. Tiene visualizaciones interactivas.
- **Lilian Weng The Multi-Armed Bandit Problem**: el mejor post para el prerrequisito de bandidos. Con gráficas, código y pruebas informales.

Nivel 3 Tutorial académico completo

- **WSC 2018 Monte Carlo Tree Search: A Tutorial (PDF)**: artículo tutorial publicado en el Winter Simulation Conference. Riguroso pero accesible. Cubre historia, variantes y aplicaciones.
- **UWashington Lecture 19: MCTS (PDF)**: diapositivas de clase universitaria con demostraciones informales del teorema de convergencia.
- **UQ Press MCTS en RL**: versión en libro de texto con contexto de aprendizaje por refuerzo completo.

Nivel 4 Artículos de investigación de referencia

- **Springer (2022) MCTS: A Review of Recent Modifications**: revisión sistemática de todas las variantes modernas de MCTS (PUCT, RAVE, GRAVE, progressive widening, etc.) con tabla comparativa de aplicaciones.
- **Winands Encyclopedia of MCTS (PDF)**: entrada de enciclopedia académica. La referencia más exhaustiva del estado del arte clásico.
- **Chessprogramming Wiki UCT**: detalles de implementación de UCT específicos para juegos de tablero, con variantes y trucos de rendimiento.

Prerrequisito: problema del bandido multi-brazo

- **Lilian Weng Multi-Armed Bandit** (ya citado, es el mejor)
- **Turing.com Guía de UCB**: explicación paso a paso del algoritmo UCB1 con ejemplos numéricos.
- **CSE 442 Multi-Armed Bandits (interactivo)**: visualización interactiva del dilema exploración-explotación en bandidos.

9. Resumen: el mapa conceptual del algoritmo

Fase	Pregunta	Herramienta	Hipótesis clave	Parámetro
Selección	¿Por qué rama bajar?	UCT	H1, H2	c
Expansión	¿Qué nodo añadir?	Aleatorio o heurístico	H6	$n_{\min}$
Simulación	¿Cuál es el valor?	Rollout aleatorio	H7, H8	política π
Retropropagación	¿Cómo actualizar?	Promedio acumulado	H3	

Resumen en una frase

MCTS trata cada nodo del árbol de juego como un problema de bandido multi-brazo, usa UCT para balancear exploración y explotación en cada nivel, y estima el valor de los estados desconocidos mediante simulaciones Monte Carlo sin necesitar ninguna función de evaluación escrita a mano.

Referencias principales

1. Kocsis, L., & Szepesvári, C. (2006). *Bandit based Monte-Carlo Planning*. In *European Conference on Machine Learning (ECML)*, 282–293. [**Artículo fundacional de UCT**]
2. Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47, 235–256. [**Garantías de UCB1**]
3. Lai, T. L., & Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1), 4–22. [**Cota inferior del regret**]
4. Browne, C. et al. (2012). A survey of Monte Carlo tree search methods. *IEEE TCIAIG*, 4(1), 1–43. [**La survey de referencia clásica**]
5. Silver, D. et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529, 484–489. [**AlphaGo**]
6. Silver, D. et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144. [**AlphaZero**]